

Labor 6: Das Oszilloskop als Mixed-Signal Messgerät

1. Vorbereitung

Lernziele:

-  Begriffe geklärt: Arduino, ADC, SPI, PWM
-  IDE (Integrated Development Environment) Arduino in Grundzügen beherrschen

1.1. Selbststudium

ARDUINO – sofern sie noch nicht damit gearbeitet haben (Prof. Mandl!)

Arduino Allgemein & IDE: <http://arduino.cc/en/main/software>

<http://arduino.cc/en/Guide/Environment>

SPI:

Übersicht: http://de.wikipedia.org/wiki/Serial_Peripheral_Interface bzw.

http://en.wikipedia.org/wiki/Serial_Peripheral_Interface_Bus

PWM:

Übersicht: <http://de.wikipedia.org/wiki/Pulsweitenmodulation>

Arduino: <http://arduino.cc/en/Tutorial/PWM>

<http://arduino.cc/en/Tutorial/SecretsOfArduinoPWM>

Für die optionale Übung:

Demo mit Gleichstrommotor: <https://www.youtube.com/watch?v=L2bl2A7h4sk>

1.2. Empfohlene Vorbereitung (freiwillig)

Falls sie bis jetzt noch keine Arduino Software (z.B. aus dem Sample Directory) testweise in Betrieb genommen haben sollten sie hier ein paarmal den gesamten Vorgang durchgespielt haben:

-  Arduino Entwicklungsumgebung (IDE) installieren
-  Arduino über USB anschließen

- ❓ Am PC / Notebook die IDE mit der richtigen Schnittstelle (Comx) konfigurieren
- ❓ Test Programm (<http://arduino.cc/en/Main/TestCode>) starten und LED kontrollieren

Allgemein: Datenblatt des HDO 6104:

http://cdn.teledynelecroy.com/files/pdf/hdo6000_oscilloscope_datasheet.pdf

Bauteile aus dem Starterkit für die Laborübung vorbereiten

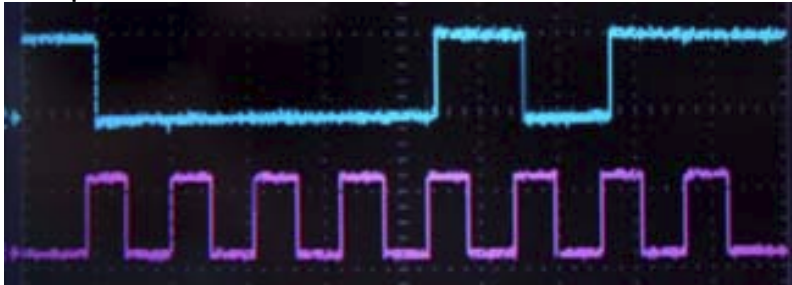
1.3. Laborvorbereitung

Folgende Fragestellungen sollten sie in kurzen Stichworten beantworten:

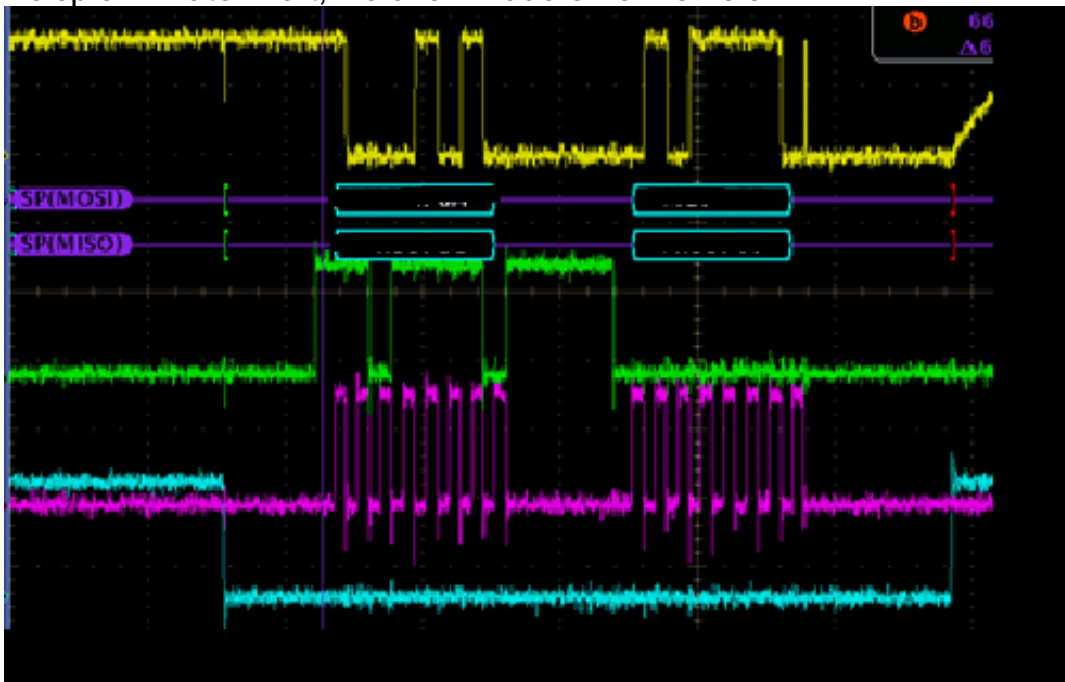
Aufgabe SPI:

Bitte jeweils das Datenwort (HEX) bestimmen, LSB first:

Beispiel1: Datenwort, welchen Mode erkennen sie



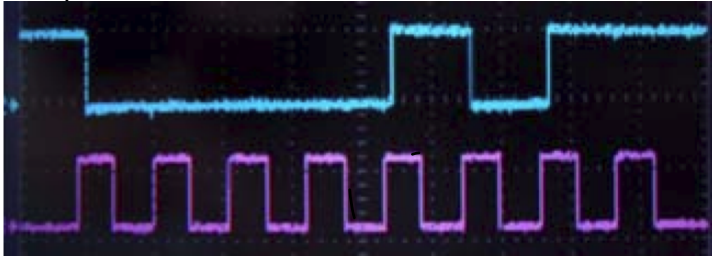
Beispiel2: Datenwort, welchen Mode erkennen sie



Gelb: MOSI, Grün: MISO

Bitte jeweils das Datenwort (HEX) bestimmen, LSB first:

Beispiel1: Datenwort, welchen Mode erkennen sie



0x50

0000 1010

Mode 1

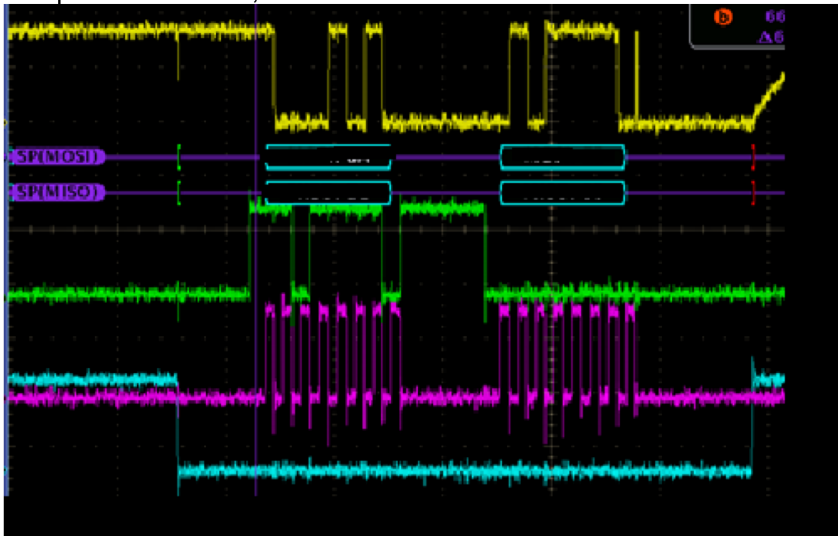
Sample $\rightarrow$ Falling Edge

CPOL = 0

Flip $\rightarrow$ Rising Edge

CPHA = 1

Beispiel2: Datenwort, welchen Mode erkennen sie



Gelb: MOSI, Grün: MISO

Flip $\rightarrow$ Falling Edge

CPOL = 0

Sample $\rightarrow$ Rising Edge

CPHA = 0

MOSI:

Mode 0

1000 1010 0101 1110

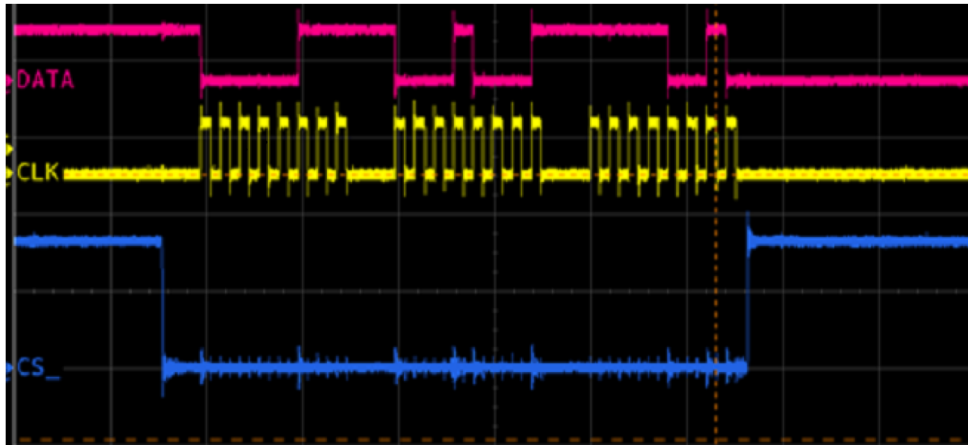
0x51

0x7A

MISO:

1101 1110 0000 0000

0x7B 0x00



Flip → Rising Edge

Sample → Falling Edge

CPOL = 0

CPHA = 1

Mode = 0

0000 0111

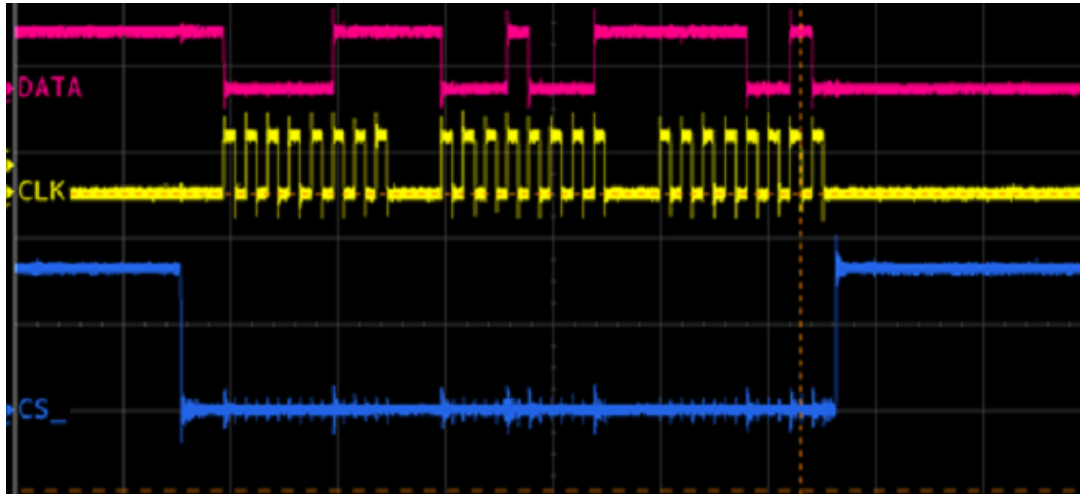
0xDD

0001 0001

0x88

1111 0010

0x4F



Aufgabe 2:

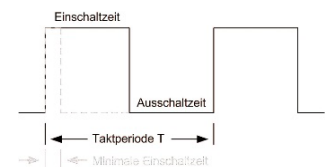
ADC 10 Bit, 0-5V, Sie messen eine Spannung von 3,75V – welchen Dezimalen / Hexadezimalen Werte erwarten sie.

Wie könnten sie die Genauigkeit der Messung erhöhen: Methode

ADC wie oben: Sie erhalten einen Hexwert von AAA – welche Spannung messen sie.

Aufgabe 3:

$$d = \frac{te}{T} * 100\%$$



Mit d = duty cycle in %, te = Zeit Aktiv / Ein (=variabel), T = Periodendauer des Signals (=konst.)

Zeichnen sie (Maßstab wählen) folgendes PWM Signal:

$f=13,6\text{MHz}$, $d=23\%$ (Frequenz die in NFC Anwendungen verwendet wird)

$f=125\text{kHz}$, $d=75\%$ (LF RFID Freq. die im Bereich z.B. Tiertracking eingesetzt wird)

Know How

Was sagt der Begriff Jitter, erklären sie ihn am Beispiel PWM, wie könnte man ihn mit einem Oszi messen

Aufgabe 2:

ADC 10 Bit, 0-5V, Sie messen eine Spannung von 3,75V – welchen Dezimalen / Hexadezimalen Werte erwarten sie.

Wie könnten sie die Genauigkeit der Messung erhöhen: Methode

ADC wie oben: Sie erhalten einen Hexwert von AAA – welche Spannung messen sie.

$$2^{10} = 1024 \text{ Zustände}$$

$$\frac{1024}{5V} \cdot 3,75V = 768_{10} \Rightarrow \text{Dezimalwert}$$

0x300

$\Rightarrow$ Durch Erhöhung der Auswertenden Bits;
je nach ADC-Typ mehr oder weniger
aufwändig.

$$0xAAA \Rightarrow 10 \cdot 1 + 10 \cdot 16 + 10 \cdot 256 = 2730 ?$$

$\Rightarrow$ Mehr als mit 10 bit darstellbar

$$5V ? \text{ oder } \frac{5V}{1024} \cdot 2730 = 13,330V \quad \text{⚡}$$

ADC 0-5V

Aufgabe 3:

$$d = \frac{t_e}{T} \cdot 100\%$$

Mit d = duty cycle in %, t_e = Zeit Aktiv / Ein (=variabel), T = Periodendauer des Signals (=konst.)

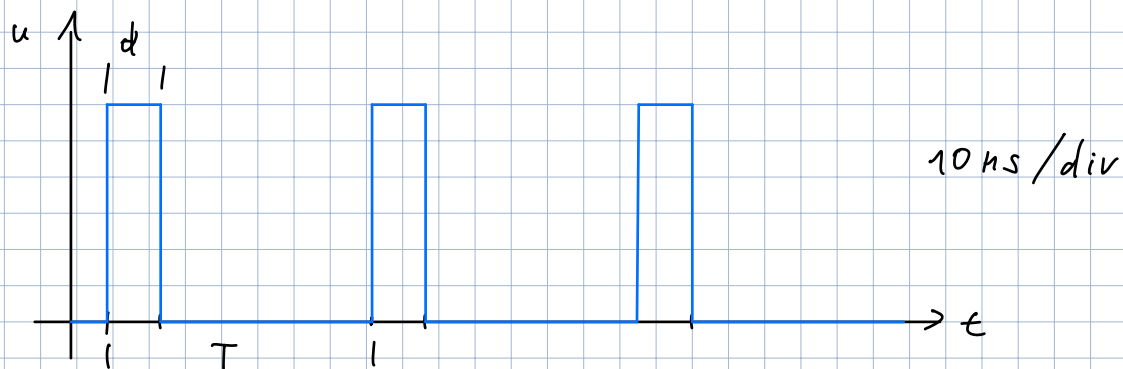


Zeichnen sie (Maßstab wählen) folgendes PWM Signal:

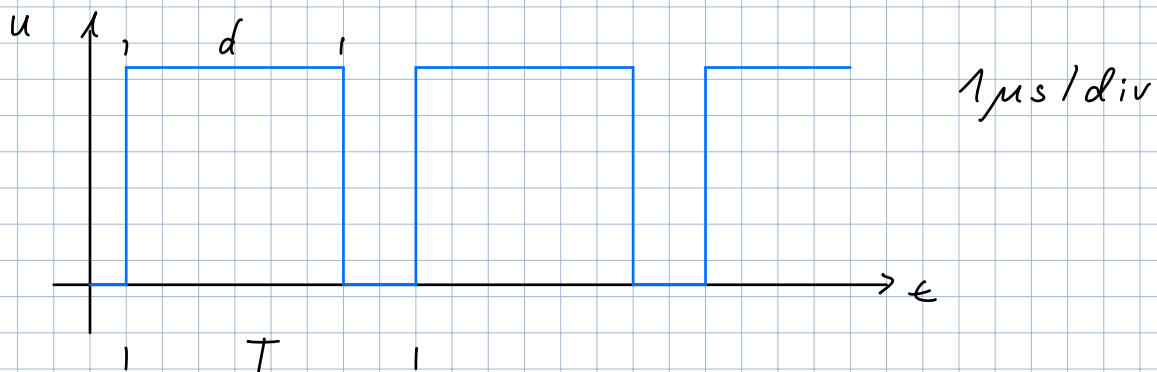
f=13,6MHz, d=23% (Frequenz die in NFC Anwendungen verwendet wird)

f= 125kHz, d= 75% (LF RFID Freq. die im Bereich z.B. Tiertracking eingesetzt wird)

a) $T = \frac{1}{f} = 73,528 \text{ ns}$ $d = 0,23 \cdot T = 16,912 \text{ ns}$

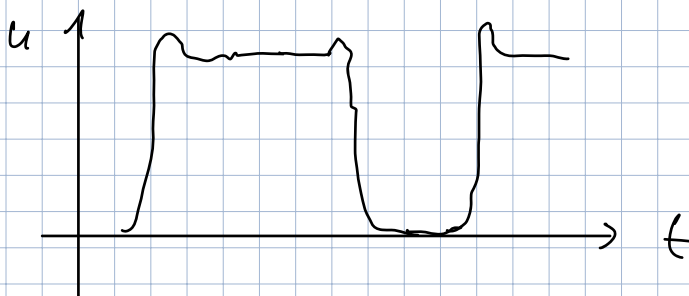


b) $T = \frac{1}{f} = 8 \mu\text{s}$ $d = 0,75 \cdot 8 \mu\text{s} = 6 \mu\text{s}$



Jitter $\rightarrow$ Taktzittern; Signal flanken werden nicht so steil und scharf wie gezeichnet in der realität aussehen

Übertriebene Darstellung:



Oszi: An Signal-
flanke zoomen

2. Laborübung

Lernziele:

- ? Mixed Signal Betrachtungen am Oszi
- ? Mixed Trigger Funktionen
- ? Arduino Handling
- ? Serieller Bus (SPI), ADC (Analog Digital Converter) und Pulsweitenmodulation (PWM) am Arduino

Für den Eingangstest:

- o Verteilen sie sich gleichmäßig im Labor (max. eine Person pro Platz) auf dem Arbeitsplatz sind ausschließlich:
- o Schreibwerkzeug (kein Federpenal)
- o Taschenrechner (aufgeladen!)
- o Leere Blätter (primär schreiben sie auf den ausgeteilten Zettel!) mit Name
- o Lineal bei Bedarf, Skizzen reichen grundsätzlich

Nicht antreffen darf man:

- o Alles andere
- o Wer beim Schummeln erwischt wird ist automatisch negativ beurteilt

Achtung: alle Sourcecode Zeilen dienen nur als Beispiel und sind unter Umständen nicht vollständig.

2.1. Gleich zu Beginn des Labors:

2.1 ist keine Messübung!

Schalten sie alle Messgeräte und den PC ein: alle IP Adressen finden sie an den jeweiligen Geräten, verbinden sie das Oszi mit dem PC (Start der Software und verbinden des Oszi)

**Während des Labors werden ihre Lab Vorbereitungen besprochen
Schreiben sie ihre Berechnungen während des Labors in diese Laborblätter und geben sie diese auf Anfrage ab (Namen nicht vergessen)**

Zu Beginn des Labors wird die Digital Probe der Oszi kurz erläutert

ACHTUNG: zur Laborübung müssen sie ihr KOMPLETTES ARDUINO DEV KIT mitbringen

Anhang: Kopie der Arduino UNO Tech Summary

2.2. Allgemeine Messungen am Arduino: (30min)

Nehmen sie ihr Arduino Board in Betrieb (wie im Auslieferungszustand, keine eigene Software läuft) und messen sie folgende Signale:

- ☐ Alle Spannungsversorgungen (Qualität, Rippel, ...)
- ☐ Quarz, Taktgeber, Oszillator, ..
- ☐ Analogen Eingänge auf 2,5V beschalten und Messen
- ☐ Kontrolle PWM Pins

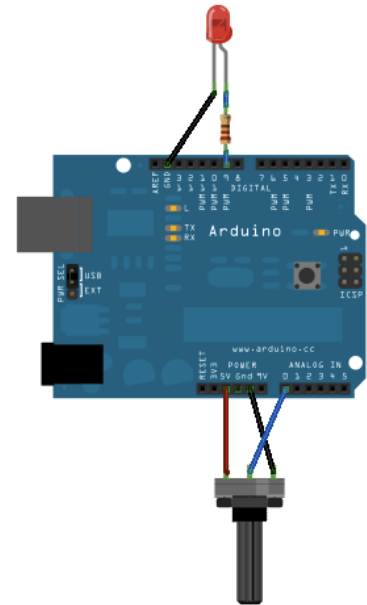
2.3. Analog In – PWM Out (1h)

Stückliste: Arduino Board, Poti 10k, LED und $R_v = 220\Omega$ (LED Vorwiderstand)

ADC: Poti – 5V, Mittelkontakt auf A0, GND

PWM: R_v auf Pin 9, LED Kathode auf GND

CycleOut: auf Pin 12



2.3.1 – Poti auf $\frac{3}{4}$ Stellung,

- ? Spannung am ADC Pin messen
- ? LED PWM vermessen, t_e , T , t_{rise} , t_{fall} ,
 - o Messen sie den Jitter des PWM Signals
- ? CycleOut messen (High -> Low,...)
- ? Serielle Schnittstelle vermessen (9600 Baud, Messvorschlag!)

Entfernen sie das Delay Statement (auskommentieren) aus dem Sourcecode

Messen sie folgende Zeiten durch Veränderung des Sourcecodes

- ? die Zeit für die AD – Wandlung
- ? die Zeit für das Senden über die serielle Schnittstelle

Machen sie einen Vorschlag wie man die LED auf einen bestimmten Helligkeitswert regeln könnte

Sourcecode zum Beispiel:

```
/*
Reads an analog input pin, maps the result to a range from 0 to 255
and uses the result to set the pulsewidth modulation (PWM) of an output pin.
Also prints the results to the serial monitor.

created 29 Dec. 2008
modified 9 Apr 2012, 30.11.2014 by C. Anselmi
by Tom Igoe

This example code is in the public domain.

*/
```

```
const int analogInPin = A0; // Analog input pin that the potentiometer is attached to
const int analogOutPin = 9; // Analog output pin that the LED is attached to
const int digCycleOut = 2; // Digital output pin that is high during the main loop cycle

int sensorValue = 0;          // value read from the pot
int outputValue = 0;          // value output to the PWM (analog out)

void setup() {
  // initialize serial communications at 9600 bps:
  Serial.begin(9600);
  pinMode(digCycleOut, OUTPUT);
}

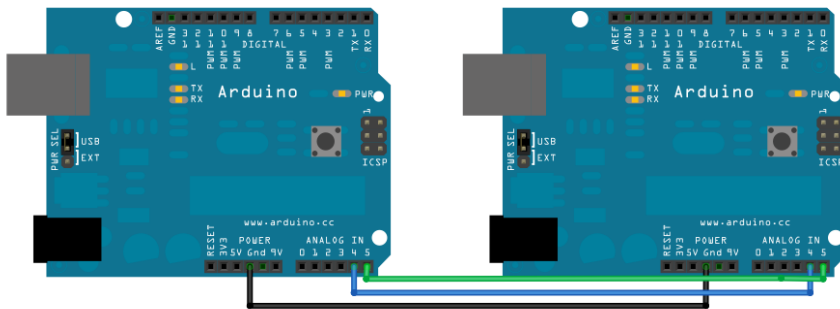
void loop() {
  digitalWrite(digCycle, HIGH);
  // read the analog in value:
  sensorValue = analogRead(analogInPin);
  // map it to the range of the analog out:
  outputValue = map(sensorValue, 0, 1023, 0, 255);
  // change the analog out value:
  analogWrite(analogOutPin, outputValue);

  // print the results to the serial monitor:
  Serial.print("sensor = ");
  Serial.print(sensorValue);
  Serial.print("\t output = ");
  Serial.println(outputValue);
  digitalWrite(digCycle, LOW);

  delay(1);
}
```

2.4. I2C Demo

Zwei Arduino Boards (jeder bringt sein eigenes mit, daher hat jede Gruppe zumindest 2 Boards!) kommunizieren über I2C: Board 1 ist Master, Board2 ist Slave



- ? Verbinden sie die richtigen Pins miteinander: SCL = Pin5, SDA = Pin4
- ? Beide Boards haben den selben Ground!
- ?

Aufgabe: Vermessen der gesamten Kommunikation

Master:

```
#include <Wire.h>
```

```
void setup() {
  Wire.begin(); // join i2c bus (address optional for master)
  Serial.begin(9600); // start serial for output
}
```

```
void loop() {
  Wire.requestFrom(8, 6); // request 6 bytes from slave device #8
```

```
  while (Wire.available()) { // slave may send less than requested
    char c = Wire.read(); // receive a byte as character
    Serial.print(c); // print the character
  }
```

```
  delay(500);
}
```

Slave:

```
#include <Wire.h>
```

```

void setup(){
  Wire.begin(8);           // join i2c bus with address #8
  Wire.onRequest(requestEvent); // register event
}

void loop(){
  delay(100);
}

// function that executes whenever data is requested by master
// this function is registered as an event, see setup()
void requestEvent(){
  Wire.write("hello"); // respond with message of 6 bytes
  // as expected by master
}

```

Details entnehmen sie auch: <https://www.arduino.cc/en/Tutorial/MasterReader>

Alles zum Thema I2C unter: <https://www.arduino.cc/en/Reference/Wire>

2.5. OPTIONAL Pinwheel Demo

Basierend auf Übung 9 des Arduino Starterkits!

Skizzieren sie die Schaltung: MOSFET (Leistungsschalter / -transistor für Motor)
 Gate Pin an Arduino Pin 9, Source Pin an GND, Drain Pin auf den Motor, 2. Motor
 Anschluss auf den Pluspol der 9Vv Batterie, Minus Pol der Batterie auf GND, Parallel
 zum Motor eine Diode (Anode auf Batterie Plus, Kathode auf Drain vom MOSFET).

MOSFET Datenblatt (Pin Belegung): siehe ilias Labor 8 oder Internet

```

const int switchPin = 2;
const int motorPin = 9;
int switchState = 0;

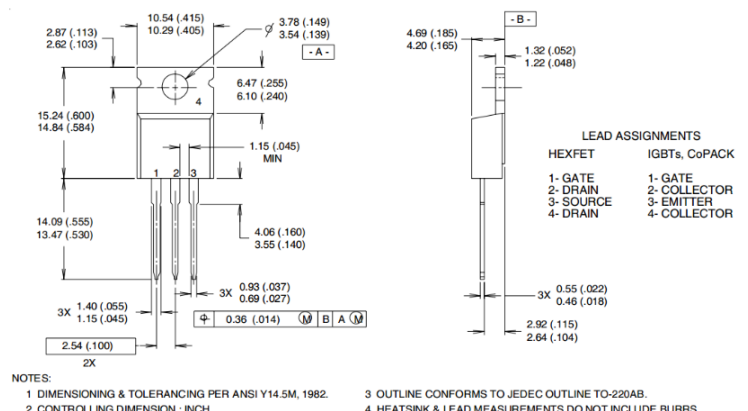
void setup() {
  pinMode(motorPin, OUTPUT);
  pinMode(switchPin, INPUT);
}

void loop() {
  switchState = digitalRead(switchPin);

  if (switchState == HIGH) {
    digitalWrite(motorPin, HIGH);
  }
  else {
    digitalWrite(motorPin, LOW);
  }
}

```

TO-220AB Package Outline
 Dimensions are shown in millimeters (inches)



Messen sie im Betrieb folgende Schaltvorgänge:

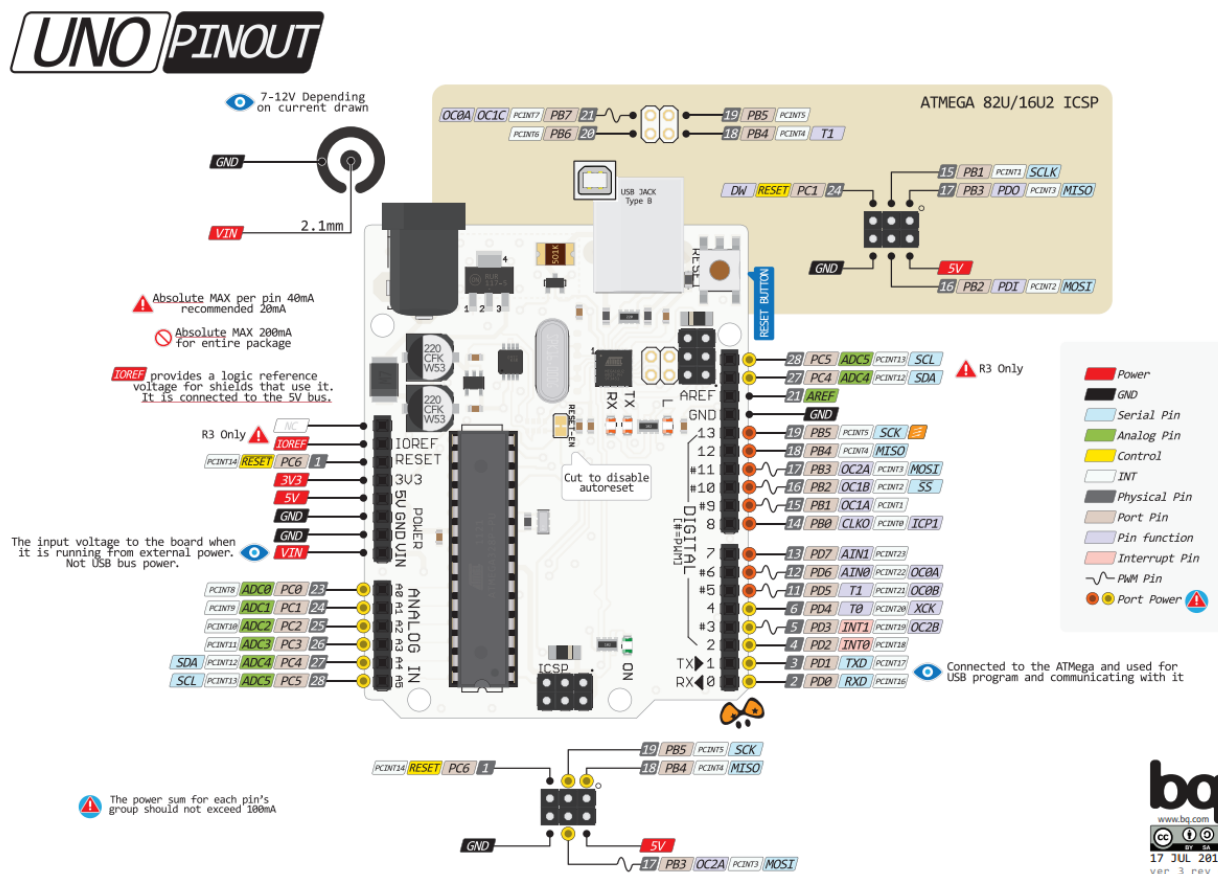
- ☐ Einschaltvorgang
- ☐ Ausschaltvorgang: Spannung am MOSFET, Motor, Diode

Wie könnte man den Strom messen der durch den Motor fließt.

Machen sie einen Vorschlag wie man die Geschwindigkeit einstellen könnte
Welche Methoden kennen sie dazu bereits.

Tech summary Arduino Uno:

- | | | |
|---|-----------------------------|--|
| ? | Microcontroller | ATmega328 |
| ? | Operating Voltage | 5V |
| ? | Input Voltage (recommended) | 7-12V |
| ? | Input Voltage (limits) | 6-20V |
| ? | Digital I/O Pins | 14 (of which 6 provide PWM output) |
| ? | Analog Input Pins | 6 |
| ? | DC Current per I/O Pin | 40 mA |
| ? | DC Current for 3.3V Pin | 50 mA |
| ? | Flash Memory | 32 KB (ATmega328) of which 0.5 KB used by bootloader |
| ? | SRAM | 2 KB (ATmega328) |
| ? | EEPROM | 1 KB (ATmega328) |
| ? | Clock Speed | 16 MHz |



Power

The Arduino Uno can be powered via the USB connection or with an external power supply. The power source is selected automatically.

External (non-USB) power can come either from an AC-to-DC adapter (wall-wart) or battery. The adapter can be connected by plugging a 2.1mm center-positive plug into the board's power jack. Leads from a battery can be inserted in the Gnd and Vin pin headers of the POWER connector.

The board can operate on an external supply of 6 to 20 volts. If supplied with less than 7V, however, the 5V pin may supply less than five volts and the board may be unstable. If using more than 12V, the

voltage regulator may overheat and damage the board. The recommended range is 7 to 12 volts.

The power pins are as follows:

VIN. The input voltage to the Arduino board when it's using an external power source (as opposed to 5 volts from the USB connection or other regulated power source). You can supply voltage through this pin, or, if supplying voltage via the power jack, access it through this pin.

5V. This pin outputs a regulated 5V from the regulator on the board. The board can be supplied with power either from the DC power jack (7 - 12V), the USB connector (5V), or the VIN pin of the board (7-12V). Supplying voltage via the 5V or 3.3V pins bypasses the regulator, and can damage your board. We don't advise it.

3V3. A 3.3 volt supply generated by the on-board regulator. Maximum current draw is 50 mA.

GND. Ground pins.

IOREF. This pin on the Arduino board provides the voltage reference with which the microcontroller operates. A properly configured shield can read the IOREF pin voltage and select the appropriate power source or enable voltage translators on the outputs for working with the 5V or 3.3V.

Memory

The ATmega328 has 32 KB (with 0.5 KB used for the bootloader). It also has 2 KB of SRAM and 1 KB of EEPROM (which can be read and written with the EEPROM library).

Input and Output

Each of the 14 digital pins on the Uno can be used as an input or output, using `pinMode()`, `digitalWrite()`, and `digitalRead()` functions. They operate at 5 volts. Each pin can provide or receive a maximum of 40 mA and has an internal pull-up resistor (disconnected by default) of 20-50 kOhms. In addition, some pins have specialized functions:

Serial: 0 (RX) and 1 (TX). Used to receive (RX) and transmit (TX) TTL serial data. These pins are connected to the corresponding pins of the ATmega8U2 USB-to-TTL Serial chip.

External Interrupts: 2 and 3. These pins can be configured to trigger an interrupt on a low value, a rising or falling edge, or a change in value. See the `attachInterrupt()` function for details.

PWM: 3, 5, 6, 9, 10, and 11. Provide 8-bit PWM output with the `analogWrite()` function.

SPI: 10 (SS), 11 (MOSI), 12 (MISO), 13 (SCK). These pins support SPI communication using the SPI library.

LED: 13. There is a built-in LED connected to digital pin 13. When the pin is HIGH value, the LED is on, when the pin is LOW, it's off.

The Uno has 6 analog inputs, labeled A0 through A5, each of which provide 10 bits of resolution (i.e. 1024 different values). By default they measure from ground to 5 volts, though it is possible to change the upper end of their range using the AREF pin and the `analogReference()` function. Additionally, some pins have specialized functionality:

I2C: A4 or SDA pin and A5 or SCL pin. Support I2C communication using the Wire library.

There are a couple of other pins on the board:

AREF. Reference voltage for the analog inputs. Used with `analogReference()`.

Reset. Bring this line LOW to reset the microcontroller. Typically used to add a reset button to shields which block the one on the board.

See also the mapping between Arduino pins and ATmega328 ports. The mapping for the ATmega8, 168, and 328 is identical.

Communication

The Arduino Uno has a number of facilities for communicating with a computer, another Arduino, or other microcontrollers. The ATmega328 provides UART TTL (5V) serial communication, which is available on digital pins 0 (RX) and 1 (TX). An ATmega16U2 on the board channels this serial communication over USB and appears as a virtual com port to software on the computer. The '16U2 firmware uses the standard USB COM drivers, and no external driver is needed. However, on Windows, a .inf file is required. The Arduino software includes a serial monitor which allows simple textual data to be sent to and from the Arduino board. The RX and TX LEDs on the board will flash when data is being transmitted via the USB-to-serial chip and USB connection to the computer (but not for serial communication on pins 0 and 1).

A SoftwareSerial library allows for serial communication on any of the Uno's digital pins.

The ATmega328 also supports I2C (TWI) and SPI communication. The Arduino software includes a Wire library to simplify use of the I2C bus; see the documentation for details. For SPI communication, use the SPI library.

Programming

The Arduino Uno can be programmed with the Arduino software (download). Select "Arduino Uno" from the Tools > Board menu (according to the microcontroller on your board). For details, see the reference and tutorials.

The ATmega328 on the Arduino Uno comes preburned with a bootloader that allows you to upload new code to it without the use of an external hardware programmer. It communicates using the original STK500 protocol (reference, C header files).

You can also bypass the bootloader and program the microcontroller through the ICSP (In-Circuit Serial Programming) header using Arduino ISP or similar; see these instructions for details.

The ATmega16U2 (or 8U2 in the rev1 and rev2 boards) firmware source code is available. The ATmega16U2/8U2 is loaded with a DFU bootloader, which can be activated by:

On Rev1 boards: connecting the solder jumper on the back of the board (near the map of Italy) and then resetting the 8U2.

On Rev2 or later boards: there is a resistor that pulling the 8U2/16U2 HWB line to ground, making it easier to put into DFU mode.

You can then use Atmel's FLIP software (Windows) or the DFU programmer (Mac OS X and Linux) to load a new firmware. Or you can use the ISP header with an external programmer (overwriting the DFU bootloader). See this user-contributed tutorial for more information.

Automatic (Software) Reset

Rather than requiring a physical press of the reset button before an upload, the Arduino Uno is designed in a way that allows it to be reset by software running on a connected computer. One of the hardware flow control lines (DTR) of the ATmega8U2/16U2 is connected to the reset line of the ATmega328 via a 100 nanofarad capacitor. When this line is asserted (taken low), the reset line drops long enough to reset the chip. The Arduino software uses this capability to allow you to upload code by simply pressing the upload button in the Arduino environment. This means that the bootloader can have a shorter timeout, as the lowering of DTR can be well-coordinated with the start of the upload.

This setup has other implications. When the Uno is connected to either a computer running Mac OS X or Linux, it resets each time a connection is made to it from software (via USB). For the following half-second or so, the bootloader is running on the Uno. While it is programmed to ignore malformed data

(i.e. anything besides an upload of new code), it will intercept the first few bytes of data sent to the board after a connection is opened. If a sketch running on the board receives one-time configuration or other data when it first starts, make sure that the software with which it communicates waits a second after opening the connection and before sending this data.

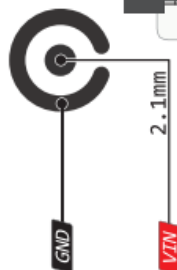
The Uno contains a trace that can be cut to disable the auto-reset. The pads on either side of the trace can be soldered together to re-enable it. It's labeled "RESET-EN". You may also be able to disable the auto-reset by connecting a 110 ohm resistor from 5V to the reset line; see this forum thread for details.

USB Overcurrent Protection

The Arduino Uno has a resettable polyfuse that protects your computer's USB ports from shorts and overcurrent. Although most computers provide their own internal protection, the fuse provides an extra layer of protection. If more than 500 mA is applied to the USB port, the fuse will automatically break the connection until the short or overload is removed.

UNO PINOUT

7-12V Depending on current drawn

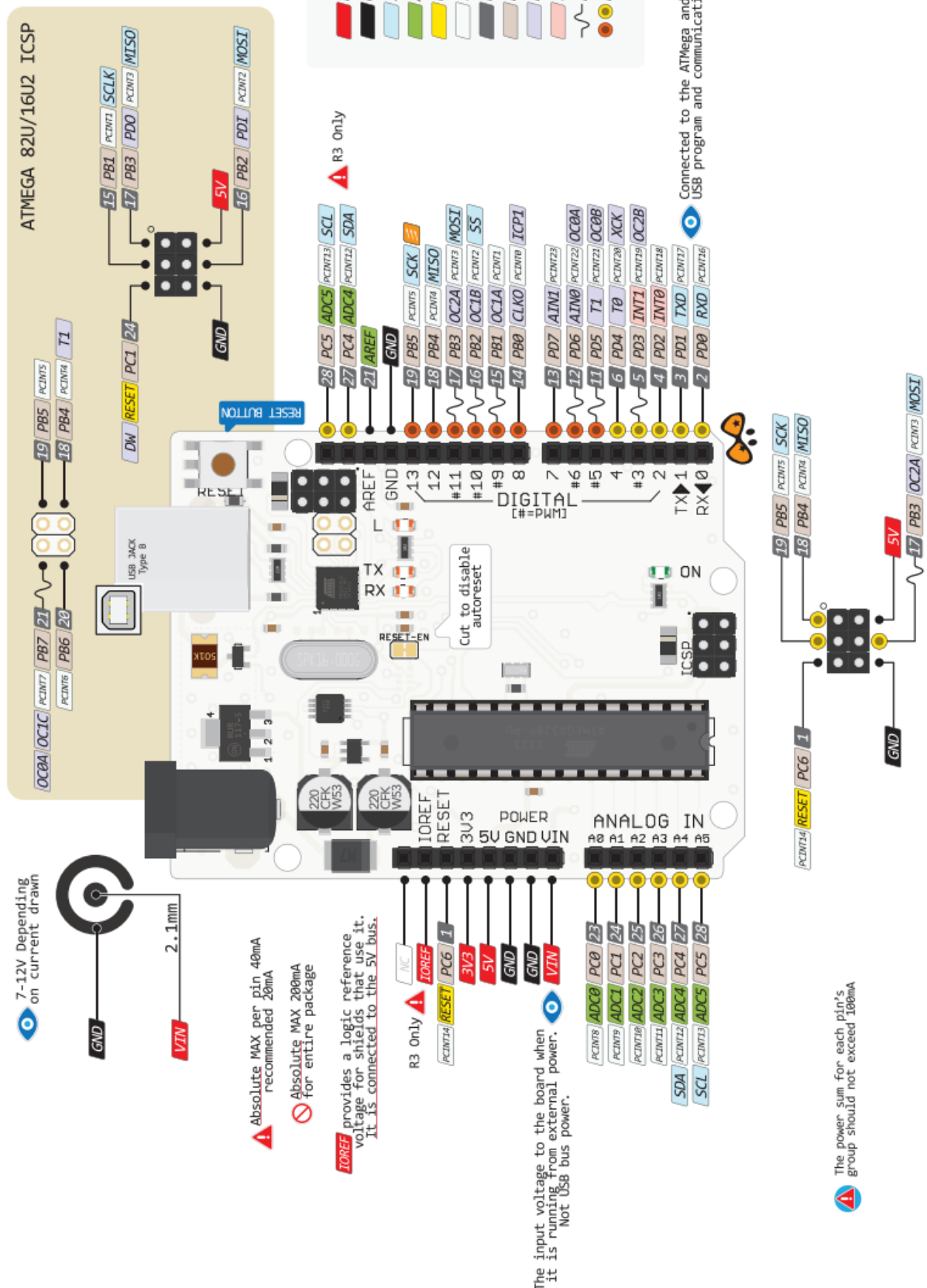


! Absolute MAX per pin 40mA recommended 20mA

! Absolute MAX 200mA for entire package

! IOREF provides a logic reference voltage for shields that use it. It is connected to the 5V bus.

The input voltage to the board when it is running from external power. Not USB bus power.



! The power sum for each pin's group should not exceed 100mA